

PROYECTO “SISTEMA DE CONTROL E INVENTARIO BIBLIOTECARIO”



1. INTRODUCCION.

Las nuevas tecnologías han eliminado la división tradicional entre lo que se conoce como archivos, bibliotecas y centros de documentación. Esta división ya obsoleta venía condicionada por los diversos soportes de la información que caracterizaban a cada una de ellas.

Así, la información es el producto final de un proceso que acumula, organiza, almacena y difunde datos, de modo que satisface las necesidades de los usuarios e incrementa sus conocimientos en un campo determinado.

Enlazando a estos conceptos, la biblioteca como sistema de información podríamos definirla como el sistema de almacenamiento y difusión de información, accesible desde cualquier ordenador conectado a la red local de la institución de la que depende o desde cualquier punto de la red de Internet.

La biblioteca como “Sistema de Información” presenta numerosas ventajas frente a los canales tradicionales, tales como:

- Toda la información se encuentra en un único lugar, estructurada y siempre accesible.
- La información es accesible desde diversos sitios geográficamente distantes.
- La actualización de la información nos permite realizarla instantáneamente.
- El acceso a la información, al disminuir el proceso informativo, nos permite aumentar la rapidez y la fiabilidad.

2. OBJETIVOS

Desarrollar un Sistema Informático vía Web y una app. Móvil, que permita gestionar los datos de una biblioteca.

3. MARCO TEORICO

3.1 Arquitecturas de diseño (o patrones de diseño)

Son patrones que proporcionan pautas y estructuras que pueden ser utilizadas para construir aplicaciones de software de manera eficiente, mantenible y escalable.

- 3.1.1 **Modelo-Vista-Controlador (MVC):** es el más utilizado en el desarrollo de aplicaciones web. Separa la lógica de la aplicación en tres componentes principales: el modelo (representa los datos y la lógica empresarial), la vista (maneja la presentación de los datos al usuario) y el controlador (coordina las interacciones entre el modelo y la vista). Algunas arquitecturas similares son: Modelo-Vista-Presentador (MVP) y Modelo-Vista-Vista de Modelo (MVVM).
- 3.1.2 **Arquitectura Cliente-Servidor (CS):** es la más antigua y ampliamente utilizada. La aplicación se divide en dos partes: el cliente, que es la interfaz de usuario y solicita servicios al servidor, y el servidor, que procesa las solicitudes del cliente y devuelve los resultados.
- 3.1.3 **Arquitectura de tres Capas (TCA):** separa la aplicación en tres capas: la capa de presentación (frontend), la capa de lógica de negocio (backend) y la capa de almacenamiento de datos (base de datos). Cada capa tiene una responsabilidad específica y se comunica con las capas adyacentes a través de interfaces y protocolos definidos.
- 3.1.4 **Arquitectura de Microservicios (MSA):** separa la aplicación en un conjunto de servicios pequeños e independientes que se ejecutan de manera independiente y se comunican entre sí a través de APIs. Cada servicio se enfoca en una tarea específica y se puede desarrollar, desplegar y escalar de forma independiente, lo que permite una mayor flexibilidad y escalabilidad en el desarrollo de aplicaciones.
- 3.1.5 **Arquitectura Orientada a Servicios (SOA):** los servicios son componentes de software autónomos y reutilizables que se comunican a través de estándares y protocolos definidos, como servicios web. SOA se centra en la interoperabilidad y modularidad, permitiendo la integración y reutilización de servicios en diferentes aplicaciones y plataformas.

3.2 Técnicas de programación

Son enfoques, métodos o prácticas utilizados por los programadores para resolver problemas y desarrollar software de manera eficiente y efectiva.

- 3.2.1 **Programación orientada a objetos (OOP):** se basa en la definición de clases y objetos, donde los objetos son instancias de clases que encapsulan datos y métodos relacionados. Promueve la reutilización de código, modularidad y estructura organizada del software.

- 3.2.2 Programación orientada a aspectos (AOP):** se utiliza para modularizar y separar las preocupaciones transversales del código principal de una aplicación. Permite encapsular y gestionar aspectos como el registro, la seguridad y el manejo de excepciones.
- 3.2.3 Desarrollo conducido por modelo (MDD):** se basa en la utilización de modelos como el principal artefacto en el proceso de desarrollo.
- 3.2.4 Desarrollo impulsado por pruebas (TDD):** se basa en escribir pruebas automatizadas antes de escribir el código de producción. Las pruebas guían el diseño y el desarrollo de la funcionalidad, lo que conduce a un código más limpio y de mayor calidad.

Spring Boot.

Spring nace de la complejidad que se tenía que hacer a la hora de crear proyectos java enterprise edition, los EJBs eran muy complejos y la forma en que se realizaba el despliegue del proyecto era muy engorroso.

Spring Boot contiene una infraestructura ligera que elimina la mayor parte del trabajo de configurar las aplicaciones basadas en Spring. El objetivo de Spring Boot es proporcionar un conjunto de herramientas para construir rápidamente aplicaciones de Spring que sean fáciles de configurar.

Spring Boot es una tecnología que nos permite crear aplicaciones autocontenidas, con esto nos podemos olvidar de la arquitectura y enfocarnos únicamente en desarrollo, delegando a Spring Boot labores como configuración de dependencias, desplegar nuestro servicio o aplicación a un servidor de aplicaciones y enfocarnos únicamente en crear nuestro código.

Para esto Spring Boot utiliza internamente un servidor de aplicaciones embebido, por defecto utiliza Tomcat, pero también lo podemos hacer con jetty o undertow.

Y no solo esto, Spring Boot también nos provee un completo gestor de dependencias como maven o gradle, configuraciones automáticas y mucho más para que nuestra aplicación quede a la medida.

Spring Boot facilita la creación de aplicaciones independientes basadas en Spring de grado de producción que puede "simplemente ejecutar".

3.3 Jpa (Java Persistence Api).

JPA es una **especificación** que indica cómo se debe realizar la persistencia (almacenamiento) de los objetos en programas Java. Fíjate en que destaco la palabra "Especificación" porque JPA no tiene una implementación concreta, sino que, como veremos enseguida, existen diversas tecnologías que implementan JPA para darle concreción.

JPA forma parte de Java empresarial desde su versión 5, en el año 2006. Desde entonces han aparecido varias versiones de la especificación, que sigue el [proceso de la comunidad Java](#), siendo la más reciente la 2.2, aparecida en el verano de 2017.

Aunque forma parte de Java empresarial, las implementaciones de JPA **se pueden emplear** en cualquier tipo de aplicación aislada, sin necesidad de usar ningún servidor de aplicaciones, **como una mera biblioteca de acceso a datos**.

3.4 Framework.

Existen en el mercado distintos servidores de aplicaciones propietarios y de código abierto y libre que resuelven el problema de una arquitectura de tipo enterprise. Por ejemplo hay varias marcas que implementan la especificación J2EE basadas en lenguaje Java.

Estas plataformas permiten además la integración con sistemas de terceros para alguna de sus capas, por ejemplo la de persistencia. Entre ellas podemos mencionar Bea WebLogic, IBM Websphere, Oracle 9i AS, JBoss, etc. También Microsoft ha desarrollado una plataforma basada en el lenguaje c#, esta plataforma es .Net que también resuelve estos tipos de problemas.

3.5 J2EE

La plataforma J2EE [WEB-1] utiliza un modelo de aplicación distribuida multicapa. Las partes de una aplicación J2EE son presentados en los siguientes componentes que brinda J2EE.

- Componentes de la Capa Cliente (Client-Tier) que corren en la máquina cliente.
- Componentes de la Capa Web (Web-Tier) que corren en el servidor J2EE.
- Componentes de la Capa de Negocio (Business-tier) que corren en el servidor J2EE.
- Software de la Capa de Enterprise information system (EIS-tier) que corren en el servidor EIS.

Aunque pueda contar de 3 o 4 capas lógicas, las aplicaciones J2EE en capas son denominadas aplicaciones de 3 capas, ya que hace referencia a capas físicas, puesto que son distribuidas en 3 lugares físicos diferentes, máquina Cliente, máquina Servidor J2EE y la máquina Servidor Base de Datos.

Las aplicaciones J2EE están basadas en componentes, cada componente tiene su propia funcionalidad y permite ser ensamblado en una aplicación J2EE, Estos componentes corren y son manejados por un servidor J2EE

3.6 Framework de Persistencia.

La capa de mayor criticidad y por consiguiente en la cual más trabajo se ha desarrollado en los últimos años es la de persistencia. Debido al choque de impedancia que se produce entre los objetos del modelo de negocio y los datos persistidos en una base de datos relacional, es que esta capa requiere un tratamiento particular.

Gran parte de los productos que se han generado atacan el problema del mapeo y acceso a los datos persistentes. Algunos de los más conocidos son:

- EJB Entity beans
- JDBC
- SQLJ
- TopLink
- CocoBase
- Hibernate / nHibernate
- JPOX (JDO)
- Versant (JDO)
- OBJ
- Object Spaces

3.7 JDBC (JdbcTemplate)

La clase `JdbcTemplate` es la clase central en el paquete (package) core de **JDBC**. Simplifica el uso de JDBC ya que esta maneja la creación y liberación de recursos. Esto ayuda a evitar errores comunes como el olvidar el cerrar siempre la conexión. Se encarga de ejecutar las funciones comunes de JDBC como la creación y ejecución de statements, dejando al código de la aplicación el proveer el SQL y extraer los resultados. Esta clase ejecuta los queries (consultas) SQL, las actualizaciones (updates) y las llamadas a procedimientos almacenados (stored procedures), realizando la iteración sobre `ResultSets` y extrayendo los parametros retornados. También captura las excepciones de JDBC y las traduce en una jerarquía de excepciones más genérica e informativa definida en el paquete `org.springframework.dao`

3.8 Base de Datos PostgreSQL.

PostgreSQL es un sistema de gestión de base de datos relacional orientada a objetos y libre, publicado bajo la licencia BSD.

Como muchos otros proyectos de código abierto, el desarrollo de PostgreSQL no es manejado por una sola empresa sino que es dirigido por una comunidad de desarrolladores y organizaciones comerciales las cuales trabajan en su desarrollo. Dicha comunidad es denominada el PGDG (*PostgreSQL Global Development Group*).

3.9 Tecnologías, Protocolos y Estándares a Utilizar.

Para este proyecto se utilizarán las siguientes tecnologías:

- JDK
- Tomcat
- Servlets con Java
- Drivers de SQL
- Velocity gestor de plantillas
- Eclipse IDE
- HTML
- XML
- DOM
- JavaScript
- AJAX

3.10 Procedimientos Almacenados.

Un **procedimiento almacenado** (*stored procedure* en inglés) es un programa (o procedimiento) el cual es almacenado físicamente en una base de datos. Su implementación varía de un manejador de bases de datos a otro. **La ventaja de un procedimiento almacenado** es que al ser ejecutado, en respuesta a una petición de usuario, es ejecutado directamente en el motor de bases de datos, el cual usualmente corre en un servidor separado. Como tal, posee acceso directo a los datos que necesita manipular y sólo necesita enviar sus resultados de regreso al usuario, deshaciéndose de la sobrecarga resultante de comunicar grandes cantidades de datos salientes y entrantes.

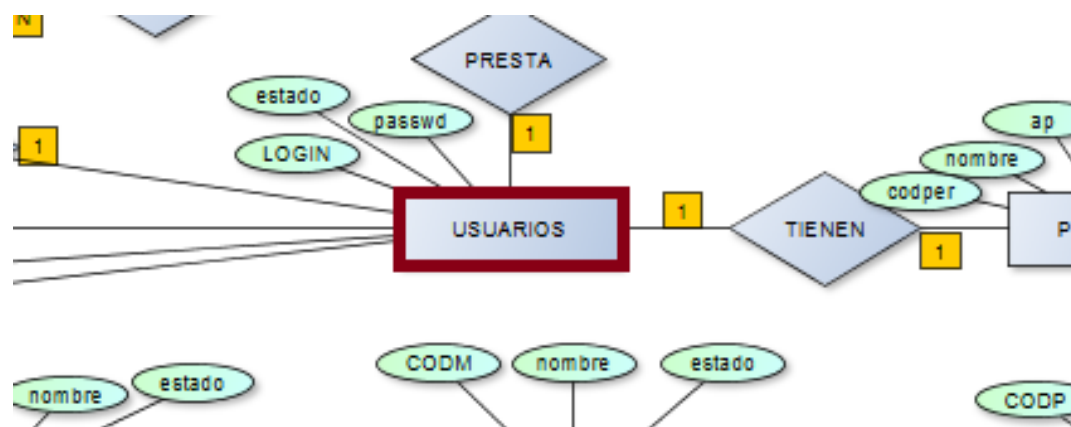
4. DESCRIPCION DEL SISTEMA

El sistema es Gestión de bibliotecas es una herramienta que permite gestionar cualquier tipo de material, ya sea impreso o digital de forma amigable y eficiente.

Este sistema informático es una herramienta flexible que satisface las necesidades de una biblioteca en cuanto al registro de los libros, registro de ejemplares, préstamos, devoluciones, registro de personas y todo lo que es seguridad del sistema.

Este sistema está diseñado completamente como una aplicación basada en WEB, permitiendo gestionar todos los servicios de la biblioteca en forma centralizada y optimizando de esta forma el flujo de trabajo.

5. MODELO CONCEPTUAL DE LA BASE DE DATOS



6. DE LA BASE DE DATOS

6.1. DICCIONARIO DE DATOS

USUARIOS.- Almacena datos de acceso al sistema de las personas.

TABLA: USUARIOS

NOMBRE	TIPO	TAM.	NULO	PK	FK	DESCRIPCIÓN
LOGIN	Texto	15	NO	SI		Nombre usuario o login del usuario
PASSWD	Texto	200	NO			Clave de acceso del usuario
ESTADO	Entero		NO			1=Activo, 0= Anulado
CODPER	Entero		NO		SI	Referencia tabla PERSONAS

PANTALLAS

7.1 B-1. ESTRUCTURA GENERAL DEL SITIO WEB



[1].- En esta área se tiene que ubicar el logo de la Biblioteca. Se sugiere que este logo sea pequeño y con un fondo que rellene el área [1] y [3].

[3].- Esta área presenta el menú principal

[2].- En esta área se ubicarán todas las páginas de este proyecto.

[Menú Principal].- Es el área donde se ubicará el menú del sistema

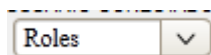
[Área de Contenidos].- Esta área se utilizará para desglosar todas las páginas de gestión.

[Usuario conectado].-En esta parte se visualiza el usuario conectado

[Fecha Literal].- Se muestra la fecha en formato literal.

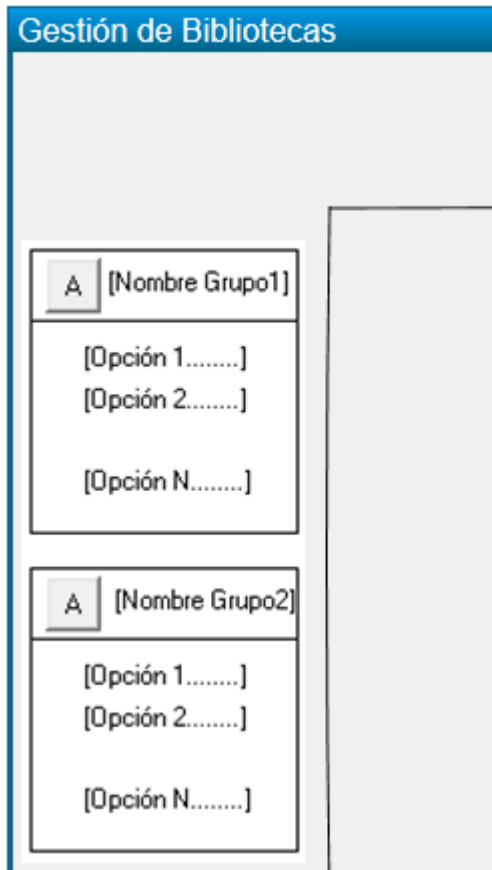


.- Icono que contiene un conjunto de opciones del usuario, este se activa al hacer un **[Click]**.



.- Lista de Roles activos de cada Usuario conectado. Cada rol seleccionado actualizará las opciones que contiene.

7.2 B-2. CONJUNTO DE MENUS



Esta pantalla muestra las cajas de opciones que corresponden a cada rol seleccionado, estas se visualizarán una vez que el usuario ingrese al sistema.

 .- Cada caja de opciones contiene un logo .

[Nombre Grupo].- Nombre de cada caja de opciones. Tabla MENUS, MEPRO.

[Opción ...].- Enlaces que hacen referencia a páginas web. Tablas MEPRO, PROCESOS.

7.3 B-3. CONECTARSE O DESCONECTARSE DEL SISTEMA



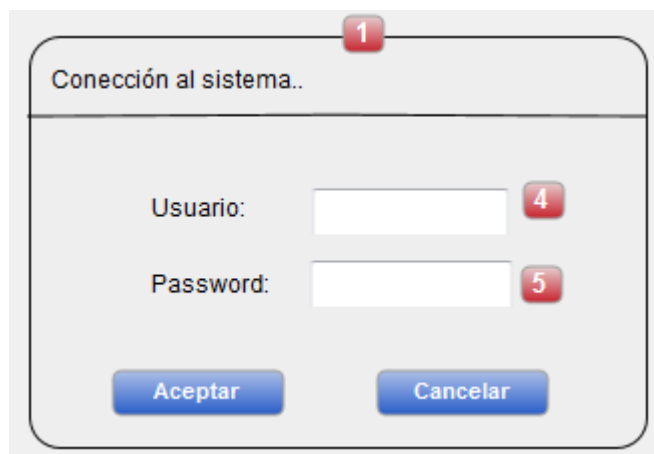
Al hacer un **[Click]** sobre el ícono del usuario se visualiza una ventana de diálogo **[4]**, el mismo que contiene las siguientes opciones.

[1].- Visualiza el Nombre y apellidos del usuario, en caso de ser un usuario invitado, visualizar “Invitado”.

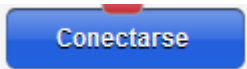
[2].-Visualiza la foto del usuario, en caso de no existir foto, se muestra una foto por defecto. También ofrece la opción de modificar la foto del usuario.

[3].-Este botón permite conectarse y si el usuario ya estuviese conectado el botón mostrará la etiqueta “Desconectar” el mismo que cerrará todas las sesiones del usuario conectado.

B-3.1. AUTENTIFICACIÓN.-



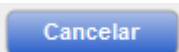
DESCRIPCION.- Esta pantalla permite autenticar a una persona que desea ingresar al sistema. Tabla USUARIOS.

ORIGEN.-El origen de esta pantalla es la opción  de **B-3**.

COMPONENTES.-

[1].- Título de la ventana

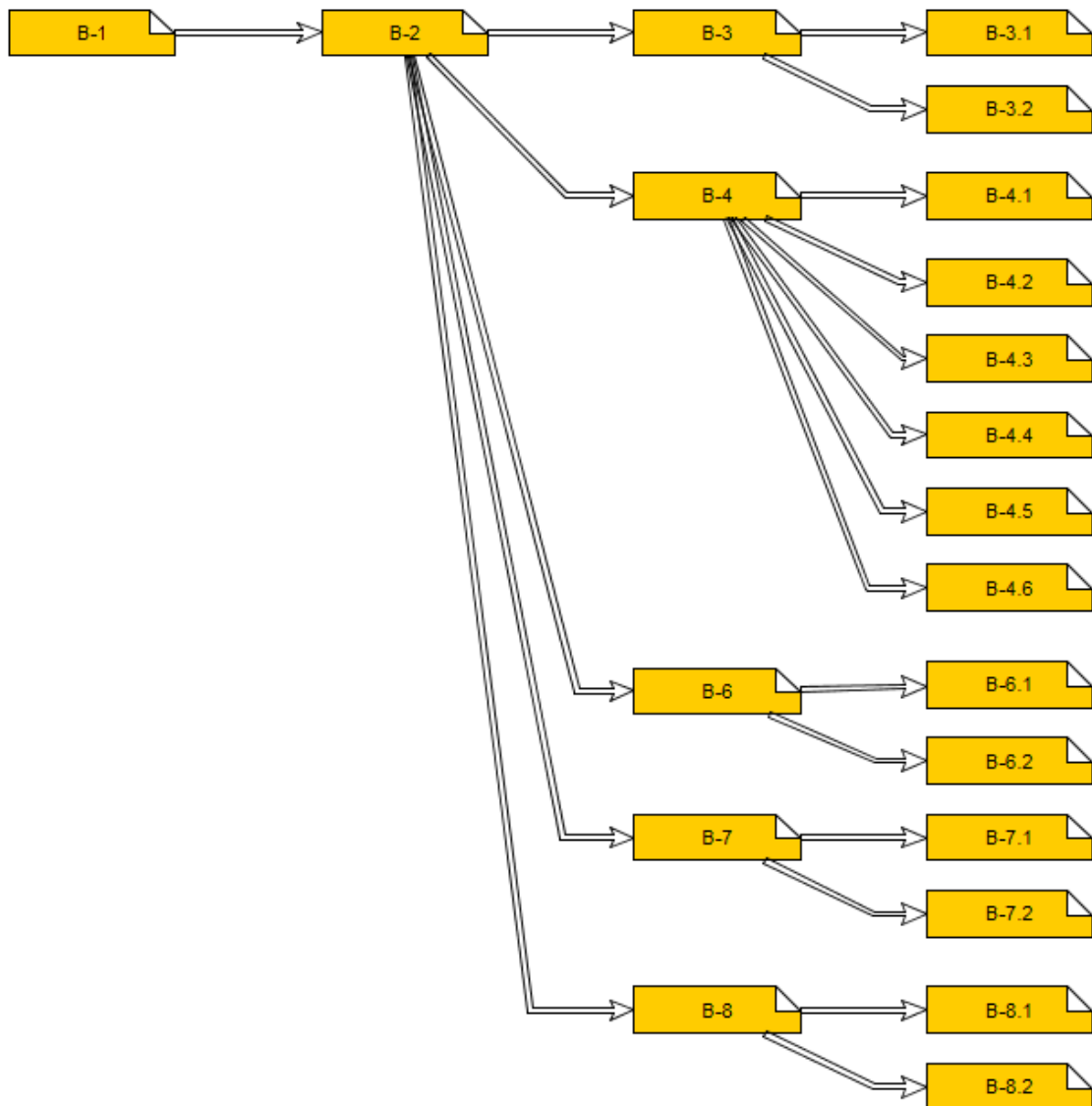
 .- Botón que permite aceptar el ingreso de datos. Antes se deberá validar los datos ingresados.

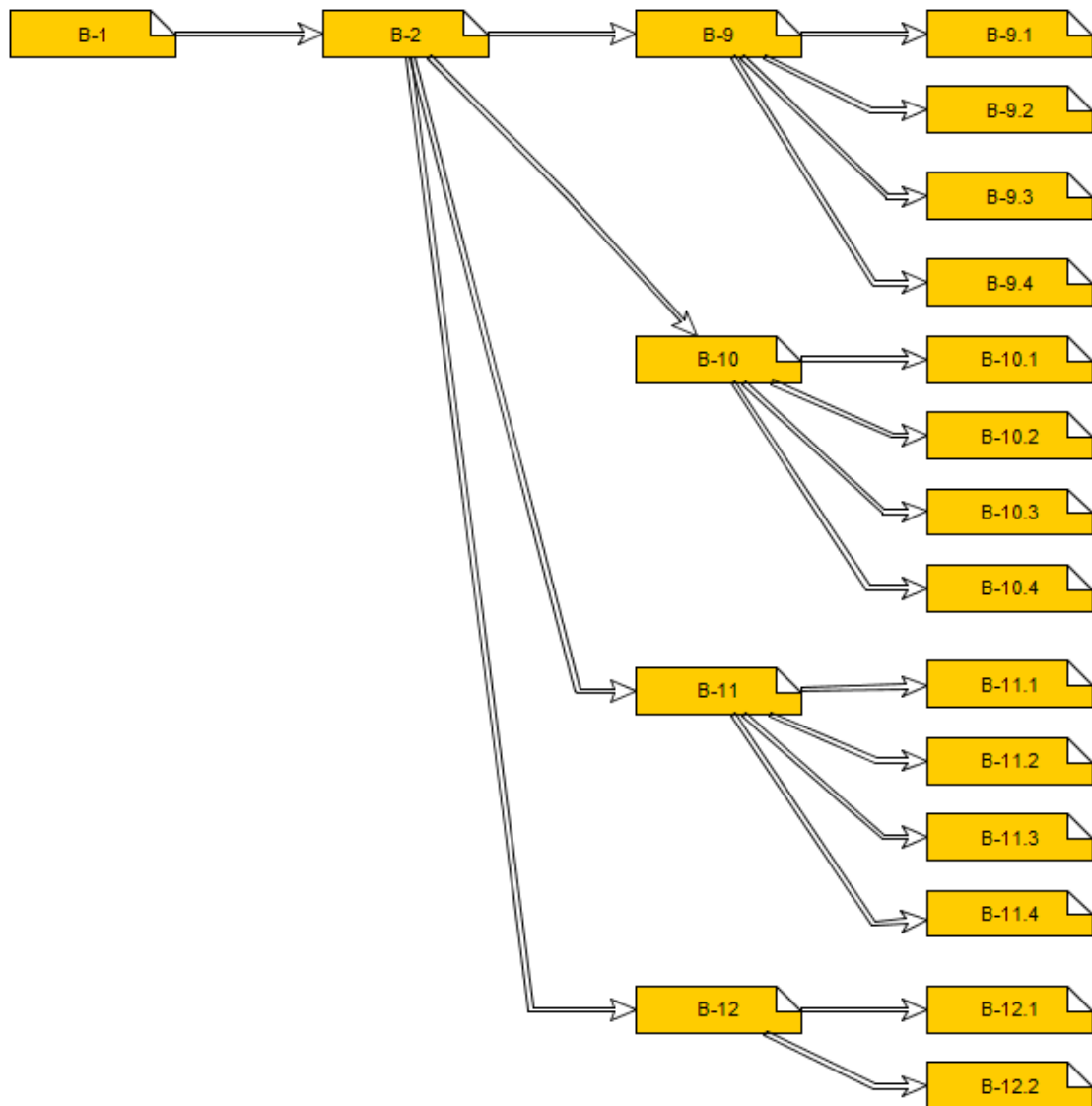
 .- Botón que permite cancelar la operación.

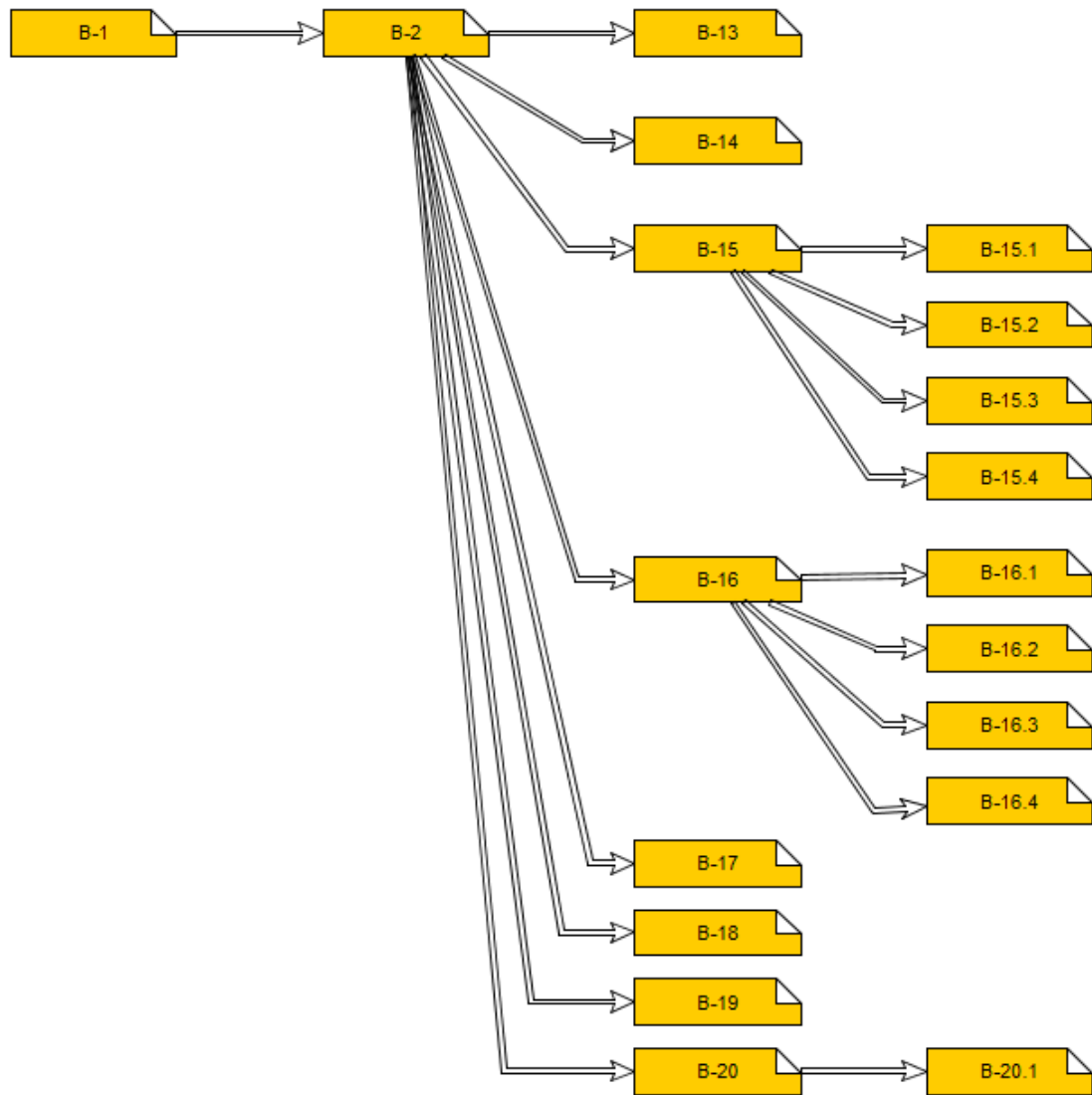
[4].- Componente de entrada, ingresa el nombre del usuario.

[5].- Componente de entrada, ingresa el password del usuario, este debe estar encriptado.

DIAGRAMA NAVEGACIONAL







CONCLUSIONES Y RECOMENDACIONES

La separación de una aplicación en capas permite separar las problemáticas y atacar los problemas de cada una en forma más independiente del resto de las capas, permitiendo así trabajar y avanzar con el desarrollo en forma paralela una vez que se especificaron las interfaces entre capas, permitiendo tener roles especializados en los temas a atacar en cada capa.

El uso de patrones ya analizados y estudiados permite hablar con más claridad y más alto nivel a los estudiantes participantes del desarrollo, permitiendo no *reinventar la rueda* y tener alternativas útiles a problemas comunes.

La utilización de *frameworks* de trabajo, que son acordes a la resolución de la problemática a resolver, facilitan el desarrollo de la aplicación ya que imponen un orden y una estructura ya pensada para un tipo de arquitectura dada. Por lo que problemas comunes y genéricos ya están resueltos y optimizados por el *framework* y se permite hacer uso de ellos y especializarlos extendiendo al mismo. Esto brinda un rápido desarrollo de la aplicación.

Lo importante es saber y distinguir claramente si un *framework*, es acorde a la problemática a resolver. Un problema típico es el uso un *framework* pensado para cierta arquitectura, cuando ésta no es la adecuada para nuestro problema, ya que dicho *framework*, fue pensado y optimizado para otra problemática.

Por esto es muy importante analizar la elección del framework adecuado.

Mediante el uso de un *framework* y en conjunto con metodologías y procesos de desarrollos adecuados, se agiliza mucho el proceso de desarrollo de una aplicación, y es más fácil obtener un resultado exitoso.

Se recomienda que los estudiantes (Taller I) que participaran del desarrollo de este sistema se acojan a las pantallas que fueron diseñados, es decir que el diseño muestra la estructura y ubicación de las opciones, por lo tanto, los estudiantes deberán implementar el sistema de acuerdo a este conjunto de pantallas, pudiendo modificar, solamente los colores, las imágenes, etc. Pero no la estructura del diseño.

Se recomienda también el **uso de procedimientos almacenados**, ya que de esta forma se aprovechan óptimamente los recursos del servidor, y se liberan procesos pesados a las máquinas clientes.